

WEEK 3 ASSIGNMENT 1

Q1. Dataset Overview

Load the dataset and answer the following:

- How many rows and columns are present?
- What are the names of all columns?
- Display the first 10 records.

```
import pandas as pd

# Load the dataset
df = pd.read_csv("agricultural_yield.csv")

# Dimensions of the dataset
print("Shape of Dataset:", df.shape)

# Display column names
print("Columns:", df.columns.tolist())

# First 10 records
print(df.head(10))
```

Q2. Data Types and Missing Values

- Check the data type of each column.
- Identify whether any missing values are present.
- If missing values exist, mention the affected columns.

```
# Check column data types and info
print(df.info())

# Check for missing values
print("Missing Values Count:")
print(df.isnull().sum())
```

Q3. Descriptive Statistics

Generate summary statistics for all numerical features and answer:

- Which feature has the highest mean value?
- Which feature has the highest standard deviation?

```
# Summary statistics  
print(df.describe())
```

Q4. Distribution Analysis

Create histograms for:

- rainfall_mm
- temperature_c
- fertilizer_kg
- yield_ton_per_hectare

Write 2–3 observations from each histogram.

```
import matplotlib.pyplot as plt  
import seaborn as sns  
  
# Plotting histograms  
features = ['rainfall_mm', 'temperature_c', 'fertilizer_kg', 'yield_ton_per_hectare']  
for col in features:  
    plt.figure(figsize=(6, 4))  
    sns.histplot(df[col], kde=True, color='forestgreen')  
    plt.title(f'Distribution of {col}')  
    plt.show()
```

Q5. Crop Type Analysis

- Find the number of records for each crop type.
- Create a count plot (bar chart) for crop_type.
- Which crop appears most frequently?

```
# Count frequency of each crop
print(df['crop_type'].value_counts())

# Count plot
sns.countplot(data=df, x='crop_type', palette='Set2')
plt.title('Crop Type Distribution')
plt.show()
```

Q6. Soil Type Analysis

- Find the frequency of each soil type.
- Create a count plot for soil_type.
- Which soil type is most common?

```
# Count frequency of each soil type
print(df['soil_type'].value_counts())

# Count plot
sns.countplot(data=df, x='soil_type', palette='Pastel1')
plt.title('Soil Type Distribution')
plt.show()
```

Q7. Yield Distribution

Create a histogram of yield_ton_per_hectare.

Answer:

- Is the distribution approximately normal?
- Are there any noticeable outliers?

```
sns.histplot(df['yield_ton_per_hectare'], kde=True, color='teal')
plt.title('Yield Distribution Analysis')
plt.show()
```

Q8. Scatter Plot Analysis

Create scatter plots of:

1. rainfall_mm vs yield_ton_per_hectare
2. fertilizer_kg vs yield_ton_per_hectare

Based on the plots:

- Which feature appears to have a stronger relationship with yield?

```
# Plot 1: Rainfall vs Yield
plt.figure(figsize=(6, 4))
sns.scatterplot(data=df, x='rainfall_mm', y='yield_ton_per_hectare', alpha=0.7)
plt.title('Rainfall vs Crop Yield')
plt.show()

# Plot 2: Fertilizer vs Yield
plt.figure(figsize=(6, 4))
sns.scatterplot(data=df, x='fertilizer_kg', y='yield_ton_per_hectare', alpha=0.7,
color='orange')
plt.title('Fertilizer vs Crop Yield')
plt.show()
```

Q9. Correlation Analysis

- Generate a correlation matrix for numerical features.
- Create a heatmap.
- Identify the top three features most correlated with crop yield.

```
# Correlation Matrix (Select only numeric columns)
corr_matrix = df.select_dtypes(include=['number']).corr()
print(corr_matrix)

# Heatmap
sns.heatmap(corr_matrix, annot=True, cmap='YlGnBu', fmt=".2f")
plt.title('Feature Correlation Heatmap')
plt.show()
```

Q10. Group-Based Analysis

Calculate the average yield for:

- Each crop type
- Each soil type

Which crop and soil type have the highest average yield?

```
# Average yield by crop type
print("Average Yield by Crop Type:")
print(df.groupby('crop_type')['yield_ton_per_hectare'].mean())

# Average yield by soil type
print("Average Yield by Soil Type:")
print(df.groupby('soil_type')['yield_ton_per_hectare'].mean())
```

Q11. Feature Encoding

The dataset contains categorical variables.

- Identify the categorical columns.
- Convert them into numerical form using One-Hot Encoding.
- Display the first five rows of the transformed dataset.

```
# Identify categorical columns (usually text columns like crop_type and soil_type)
print("Categorical Columns:", df.select_dtypes(include=['object']).columns.tolist())

# Apply One-Hot Encoding
df_encoded = pd.get_dummies(df, columns=['crop_type', 'soil_type'], dtype=int)
print(df_encoded.head())
```

Q12. Feature Selection

Separate:

- Input features (X)
- Target variable (y)

Specify which column is being used as the target variable.

```
# y is our answer/target variable
y = df_encoded['yield_ton_per_hectare']

# X contains all predictive features (drop target and identifiers)
X = df_encoded.drop(columns=['yield_ton_per_hectare'])

print("Features (X) columns:", X.columns.tolist())
print("Target variable (y): yield_ton_per_hectare")
```

Q13. Train-Test Split

Display the shape of:

Split the dataset into:

- 80% Training Data
- 20% Testing Data
- X_train
- X_test
- y_train
- y_test

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

Q14. Linear Regression Model

- Train a Linear Regression model.
- Display the model coefficients and intercept.
- Which feature has the highest positive coefficient?

```
from sklearn.linear_model import LinearRegression

# Initialize and train model
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

# Display results
print("Intercept:", lr_model.intercept_)
for col, coef in zip(X.columns, lr_model.coef_):
    print(f"{col}: {coef:.4f}")
```